Prueba técnica Full Stack — .NET + Angular + SQL Server

Contexto y objetivo

Construirás un módulo de **Bonos de desempeño** para el área de Recursos Humanos: el usuario busca un empleado, le registra un bono para un periodo (monto fijo o porcentaje del sueldo) con un motivo, y el sistema calcula el monto bruto, la retención y el neto a pagar.

El módulo tiene tres partes: base de datos en SQL Server, una API REST en .NET y una pantalla en Angular. Las tablas de áreas, empleados y estados ya existen; tú creas el resto.

Evaluamos cómo modelas datos, cómo separas responsabilidades en el backend, cómo aplicas reglas de negocio con dinero y cómo construyes una pantalla usable con paginación desde el servidor.

Instrucciones para el candidato

Tienes **24 horas** desde que recibes este documento; el trabajo estimado es de 3 a 4 horas. Prioriza que funcione y que el código sea legible antes que cubrir los puntos extra.

Recibes un **proyecto base** que ya compila y levanta: base de datos con datos de prueba, API con catálogos y manejo de errores, y front con la ruta `/bonuses`. Parte de él; no cambies el stack ni la estructura de capas.

Capa	Tecnología (ya configurada en el proyecto base)
Base de datos	SQL Server 2019+ (local, Docker o LocalDB)
Backend	.NET 10, ASP.NET Core Web API, Entity Framework Core
Frontend	Angular 22, componentes standalone; librería de UI a tu elección

Entregables

1. Repositorio Git con historial de commits; no un único commit final.
2. Tus scripts SQL en `/database`, numerados a partir de `02_`.
3. `README.md` actualizado con tus decisiones técnicas y lo que quedó pendiente.

Puedes usar cualquier recurso, incluida IA, pero en la entrevista te pediremos explicar y modificar tu código en vivo.

Base de datos

Ejecuta `database/01_schema_seed.sql` del proyecto base sin modificarlo: crea la base `BonusTest` con estas tablas y datos de prueba.

Tabla	Contenido	Clave
`State`	Estados de empleado (01xx) y de bono (02xx)	`StateCode`

Tabla	Contenido	Clave
`Department`	2 áreas: TECH1, VENTA	`DepartmentCode`
`Employee`	5 empleados con sueldo mensual y % de retención	`DepartmentCode` + `EmployeeNumber`
`EmployeeProjectHours`	Horas del último mes por proyecto de cada empleado	`EmployeeProjectHoursId`
`BonusType`	1 = Monto fijo, 2 = Porcentaje del sueldo	`BonusTypeId`
`BonusReason`	4 motivos, uno inactivo	`ReasonCode`

Lo que diseñas tú (en `02_*.sql`): la tabla `Bonus` con los campos que necesiten las reglas de negocio, sus claves foráneas —incluida la clave compuesta hacia `Employee`— y los índices que justifiques.

Reglas de negocio

Todo bono nace en **Registrado (0201)**. Solo en ese estado se puede anular (pasa a **Anulado, 0203**) o eliminar. **Pagado (0202)** solo se usa si haces el punto extra 1.

Búsqueda del empleado

1. Se busca por área + número de empleado.
2. Solo se aceptan empleados Activos (0101); un empleado Cesado devuelve un error de validación.

Registro

1. El usuario ingresa: periodo (`YYYY-MM`), tipo de bono, monto o porcentaje, y motivo (solo motivos activos).
2. El periodo no puede ser posterior al mes actual.
3. Un empleado no puede tener más de un bono activo en el mismo periodo (activo = cualquier estado distinto de Anulado y no eliminado).
4. Monto fijo: mayor a 0 y no mayor al sueldo mensual. Porcentaje: mayor a 0 y hasta 100.
5. Eliminar es lógico: el registro deja de aparecer en el listado pero no se borra de la tabla.
6. Guarda quién y cuándo creó y modificó cada registro.

Cálculo del bono

Todos los montos se redondean a 2 decimales, con el medio hacia arriba (0.125 → 0.13).

- Bruto = el monto fijo, o `MonthlySalary × porcentaje / 100`.
- Retención = `Bruto × TaxRate / 100` (el `TaxRate` es del empleado).
- Neto = Bruto − Retención.

Ejemplo: empleado TECH1-101 (sueldo 5,000.00, retención 8 %), bono de 10 % → bruto 500.00, retención 40.00, neto 460.00.

Requerimientos Backend

Agrega el módulo de bonos respetando las capas del proyecto base (`Domain`, `Application`, `Infrastructure`, `Api`); ninguna regla de negocio va en el controlador.

Ya incluido en el proyecto base: catálogos (`/api/v1/Catalog/...`), `PagedResponse<T>` y `PaginationRequest`, `BusinessException` (responde 409 con `{ code, description }`), CORS para el front y Swagger.

Método	Ruta	Qué hace
GET	`/api/v1/Bonus`	Listado paginado con filtros
GET	`/api/v1/Bonus/employee/{departmentCode}/{employeeNumber}`	Busca el empleado
POST	`/api/v1/Bonus`	Registra un bono
PUT	`/api/v1/Bonus/{bonusId}/cancel`	Anula (0201 → 0203)
DELETE	`/api/v1/Bonus/{bonusId}`	Elimina de forma lógica

Listado paginado

- Filtros opcionales: área, número de empleado, periodo y estado.
- La paginación se hace en la base de datos, no en memoria.
- Cada fila trae descripciones listas para mostrar: estado, tipo, motivo, nombre del empleado y área.

Las validaciones de las reglas de negocio se aplican en el backend aunque el front también las haga. El usuario que opera llega en el header `UserCode`.

Requerimientos Frontend

Completa la pantalla de la ruta `/bonuses` del proyecto base: un listado paginado desde el servidor y un modal para registrar. El interceptor que muestra los errores 409 ya está hecho; no repitas ese manejo en los componentes.

Pantalla principal

1. Filtros: área, número de empleado, periodo y estado (los combos ya cargan desde la API); botones Buscar y Limpiar.
2. Tabla con: acciones, estado, periodo, área y número de empleado, nombre, tipo, monto o porcentaje (con `%` cuando aplique), motivo, bruto, retención y neto.
3. Paginación de 10 filas que llama a la API en cada cambio de página y muestra el total de registros.
4. Acciones por fila: anular y eliminar, habilitadas solo en estado Registrado y con confirmación antes de ejecutar.

Modal de registro

1. Búsqueda por área + número de empleado, con validación de campos obligatorios.

2. Al encontrar al empleado, muestra nombre, fecha de ingreso, sueldo mensual y % de retención.
3. Formulario: periodo, tipo de bono, monto o porcentaje (la etiqueta cambia según el tipo) y motivo.
4. Vista previa de bruto, retención y neto mientras el usuario escribe.
5. Al guardar con éxito, cierra el modal y refresca el listado.

Puntos extra (opcionales)

Hazlos solo si la parte obligatoria ya funciona.

1. **Pagar bono y distribuirlo por proyecto.** Endpoint `POST /api/v1/Bonus/{bonusId}/pay` y botón en la grilla. Reparte el bruto entre los proyectos del empleado (`EmployeeProjectHours`), en proporción a las horas. Cada línea se redondea a 2 decimales; el último proyecto recibe lo que falta para cuadrar exacto, y si hay un solo proyecto recibe todo. Guarda el detalle y pasa el bono a Pagado (0202) en una sola transacción.
2. **Pruebas unitarias** del cálculo del bono (xUnit, NUnit o MSTest).

Criterios de evaluación

La prueba se califica sobre 100 puntos; se aprueba con 65 y los puntos extra suman hasta 10 adicionales.

Criterio	Qué se revisa	Puntos
Reglas de negocio	Estados, validaciones y cálculos dan los resultados esperados	30
Backend	Respeta las capas del proyecto base, endpoints completos, paginación en BD	20
Base de datos	Diseño de `Bonus`: tipos, nulos, claves foráneas, índices	15
Frontend	Pantalla completa, paginación desde servidor, validaciones del modal	20
Calidad de código	Nombres claros, sin duplicación, sin código muerto	10
Entrega	README, scripts en orden, historial de commits, levanta a la primera	5
Puntos extra	Punto 1: hasta 7 · Punto 2: hasta 3	hasta 10